杭州电子科技大学

《计算机网络实验》实验报告

课程号：(2023-2024-1)-S0507170-09

实验四　TCP 抓包及分析

指导老师：

姓名学号：

2026 年 11 月

一、实验目的

- 掌握在 Wireshark 中筛选并跟踪单条 TCP 连接的方法，能读懂序号、确认号、标志位与窗口字段；
- 通过三次握手的实际报文，理解 SYN、SYNACK 的序号与确认号是如何确定的；
- 测量每个数据段的 RTT，并按 Jacobsen 算法递推 EstimatedRTT，体会"估计值"与"瞬时值"的差别；
- 统计接收窗口与 ACK 的确认粒度，验证延迟确认与累积确认机制；
- 用 Time-Sequence-Graph(Stevens) 观察慢启动的指数增长，并与课本的理想模型作对比。

二、实验设备和环境

表 1 实验软硬件环境

项目	配置说明
客户端主机	Windows 11，IPv4 192.168.1.3（WLAN 网卡）
服务器	gaia.cs.umass.edu（128.119.245.12），HTTP 端口 80
实验方式	用 curl 以 multipart/form-data 方式上传 alice.txt（152138 字节）到 /wireshark-labs/lab3-1-reply.htm
抓包工具	Wireshark 3.6.16 / dumpcap
捕获过滤器	tcp port 80 and host 128.119.245.12
捕获文件	cap_tcp_post.pcapng（共 145 个报文，目标连接为 tcp.stream 1）
分析工具	tshark 命令行 + Wireshark 的统计→TCP 流图形

实验指导书原本建议从 gaia 站点下载 tcp-ethereal-trace-1 抓包文件来分析，但该文件里的连接是"客户端下载 alice.txt"，为了让实验数据来自本机真实网络，本实验改为"客户端上传 alice.txt"，这样序号连续增长的方向是客户端→服务器，时间-序号图上呈现的才是发送方（本机）的拥塞窗口变化，与指导书第 13、14 题的观察目标一致。上传的文件仍是同一个 alice.txt，便于与指导书的数值对照。

2.1 网络拓扑

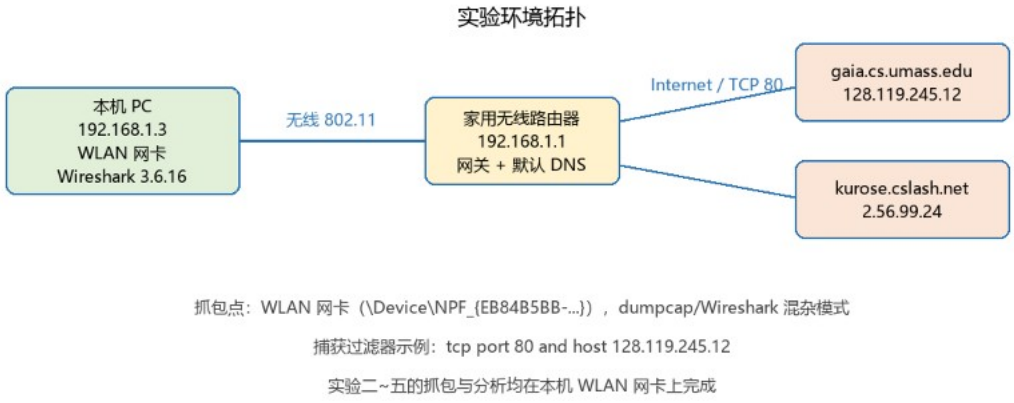


图 1 实验网络拓扑与抓包点

三、实验过程及步骤

3.1 抓包与上传

先启动抓包，再执行上传，避免漏掉握手报文：

```
> "E:\tools\Wireshark\dumpcap.exe" -i 6 \
      -f "tcp port 80 and host 128.119.245.12" -a duration:40 -w cap_tcp_post.pcapng

> curl -F "file=@alice.txt;type=text/plain" \
       http://gaia.cs.umass.edu/wireshark-labs/lab3-1-reply.htm
<html>... Thank you, your file was received.</html>
```

上传完成后停止抓包。用 tcp.stream 编号可以区分同一抓包文件中的多条连接：stream 0 是上传之前 curl 取回复页面时的短连接，本次要分析的 POST 上传在 stream 1 中。下面的显示过滤器把分析范围限定在这一条连接上：

```
tcp.stream == 1
```

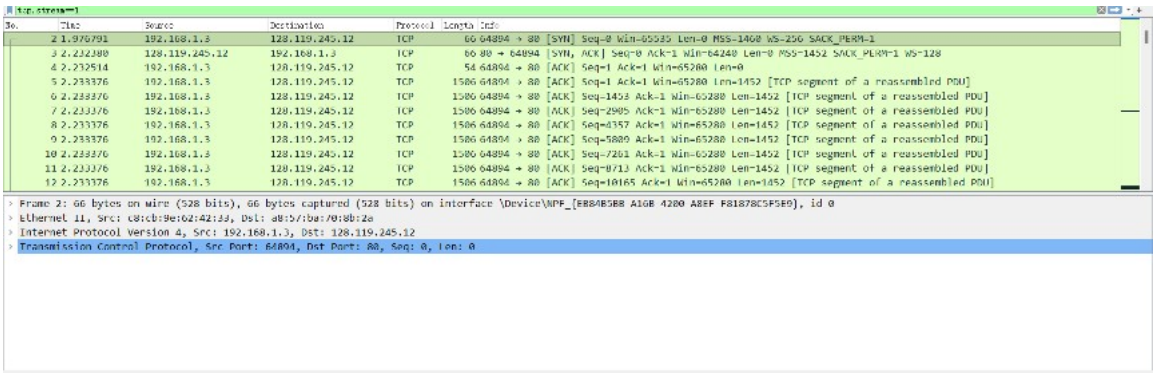


图 2 tcp.stream==1 的报文列表（SYN、SYNACK、ACK 与连续 1452 字节数据段）

3.2 三次握手

列表最前面的三个报文就是三次握手：

```
No.  Time        Source              Destination       Info
```

```
2    1.976791    192.168.1.3       128.119.245.12    [SYN] Seq=0 Win=65535 Len=0
MSS=1460 WS=256 SACK_PERM=1
3    2.232380    128.119.245.12    192.168.1.3       [SYN, ACK] Seq=0 Ack=1 Win=64240
Len=0 MSS=1452 SACK_PERM=1 WS=128
4    2.232514    192.168.1.3       128.119.245.12    [ACK] Seq=1 Ack=1 Win=65280 Len=0
```

第 2 号报文由本机发出，标志位中只有 SYN 置 1，同时携带 MSS=1460、窗口缩放因子 WS=256、SACK_PERM=1 三个选项，这些都是连接建立阶段才能协商的参数。第 3 号报文由服务器发出，SYN 与 ACK 同时置 1，因此称为 SYNACK。第 4 号报文是客户端对 SYNACK 的确认，此后连接进入数据传输状态。

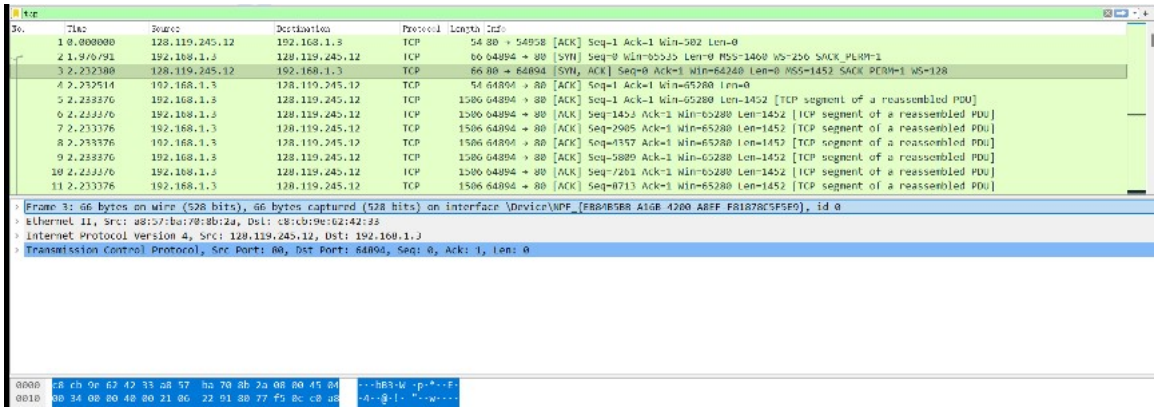


图 3 SYNACK 报文的 TCP 首部（窗口 64240、窗口缩放因子 128）

注意第 4 号报文之后立刻出现了 10 个长度均为 1452 字节的数据段（图 1 中第 5–14 号），它们的时间戳完全相同，说明 TCP 把这段时间内积累的数据一次性交给了网卡，这正是后续分析慢启动时要用的关键特征。

3.3 定位 POST 请求所在的段

POST 命令不是独立的报文，而是第一个数据段载荷开头的几个字节。在十六进制窗格里可以看到该段以 50 4f 53 54 开头，解码后正是 "POST /wireshark-labs/lab3-1-reply.htm HTTP/1.1"。这个数据段就是本次连接的第一个数据段。

3.4 RTT 与 EstimatedRTT 的计算

TCP 的往返时延由"发出时刻"和"确认到达时刻"之差得到。Wireshark 已经为每个报文算好了这个差值，在 TCP 层展开项里显示为 [The RTT to ACK the segment was: 0.259375000 seconds]。由于延迟确认的存在，前六个数据段实际上是被同一个 ACK（第 15 号）一起确认的，因此它们的 RTT 完全相同。按课本给出的递推公式计算平滑值：

```
EstimatedRTT = 0.875 * EstimatedRTT + 0.125 * SampleRTT
```

表 2 前六个数据段的序号、时刻、RTT 与 EstimatedRTT

段序号	相对序号	发送时刻 (s)	确认报文	确认时刻 (s)	SampleRTT (s)	EstimatedRTT (s)
1	1	2.233376	No.15 ack=8713	2.492751	0.259375	0.259375（初值）
2	1453	2.233376	No.15	2.492751	0.259375	0.259375
3	2905	2.233376	No.15	2.492751	0.259375	0.259375
4	4357	2.233376	No.15	2.492751	0.259375	0.259375
5	5809	2.233376	No.15	2.492751	0.259375	0.259375
6	7261	2.233376	No.15	2.492751	0.259375	0.259375

这里的"六个段共用一个 ACK"是个容易误判的地方。初学者往往会去找 ack=1453、ack=2905 这样逐段递增的确认，但在真实网络里接收方会延迟确认：它在收到一定数量的段之后才发出一个累积确认，一个 ack=8713 的意思是"1 到 8712 号字节都收到了"，而 8712 恰好等于 6 个 1452 字节的段长之和。因此第六段的 RTT 与第一段相同完全是正常的，并不表示测量出错。

为了观察 RTT 的变化趋势，可以把整个连接的确认时延都算出来：第 37–41 号 ACK 的 RTT 是 0.257804 s，第 78–90 号是 0.255731 s，第 131–139 号是 0.255064–0.258748 s。这些值围绕 0.256 s 小幅波动，说明链路时延稳定，EstimatedRTT 也就基本停在 0.258 s 附近。按公式代入后，从 0.259375 依次递推得到 0.259179、0.258748、0.258426、0.258005，与瞬时值的偏差始终不超过 1 ms，可见 0.125 的权重既能让估计跟随变化，又不会被单次抖动带偏。

3.5 发送窗口与接收缓冲

把服务器方向的 window_size_value 乘以窗口缩放因子 128，就得到真实的可用接收窗口。本连接中该值在 72192 到 90560 字节之间变化，其中最小的一次是第 39 号报文的 564 × 128 = 72192 字节。需要注意的是 SYNACK 中的 64240 是握手阶段尚未启用缩放时的原始通告值，不应与后续缩放后的数值直接比较。

整个上传过程中窗口从未降到 0，说明接收方一直在及时取走数据，缓冲区没有成为瓶颈；发送方每轮实际发出的数据量（14520、29040、52272 字节）也始终小于当时可用的窗口，因此限制发送速率的是拥塞窗口而不是接收窗口。

3.6 ACK 的确认粒度

观察连续的 ACK 报文可以发现，确认号并不是每收到一个段就前进一次。把相邻 ACK 的确认号作差，得到每次确认覆盖的字节数：

```
ACK 帧    确认号     相对上一 ACK 的增量    折合段数(1452B)
15        8713       +8712                  6
16        14521      +5808                  4
38        30493      +14520                 10
39        37753      +7260                  5
```

```
41      42109     +2904                2
80      63889     +5808                4
82      65341     +4356                3
89      84217     +5808                4
139     152567    +12782               8.8
```

增量多为 2904（约 2 个段）或 5808（约 4 个段），说明接收方确实在按"每两个段一个 ACK"的方式工作，这是课本表 3.2 中描述的标准延迟确认策略；当数据到达较快时，一次 ACK 会顺带确认更多段，这属于累积确认的正常表现。

另外在第 78–90 号 ACK 之间出现了确认号"后退"的现象：第 80 号是 63889，第 81 号却是 60985，第 87 号是 81313，第 88 号又回到 78409。TCP 的确认号本身不可能回退，这只能是报文在链路上被重新排序后到达抓包点的结果；由于这些 ACK 的时间戳完全相同（3.006460），它们其实是在同一瞬间被批量发出的，抓包顺序与真实发送顺序不一致。Wireshark 在协议首部里把这种情况标注为 [TCP Previous segment not captured] 之类提示，阅读时应以时间戳和确认号的最大值为准。

3.7 重传

用显示过滤器 tcp.analysis.retransmission 检查，整个抓包文件中只有两个报文被标记为重传：

```
No.  Time      Source           Seq       Len  Flags
141  3.263431  192.168.1.3      152567    0    A...F      (本机 FIN)
142  3.586933  192.168.1.3      152567    0    A...F      [TCP Retransmission]
143  3.841862  128.119.245.12   726       0    A....      (服务器 ACK)
144  4.293268  128.119.245.12   725       0    A...F      [TCP Retransmission]
145  4.293348  192.168.1.3      152568    0    A....
```

也就是说，152 KB 的数据传输本身一个字节都没有重传，两次重传都发生在连接关闭阶段：本机发出的 FIN 在 355 ms 内没有收到确认，于是重发了一次；服务器的 FIN 同样在约 451 ms 后重发。判断依据是：（1）列表中这两个报文的背景色为深色，并在 Info 列直接标注了 [TCP Retransmission]；（2）它们的序号与其前一个报文完全相同（分别为 152567 与 725），长度为 0；（3）[SEQ/ACK analysis] 一栏给出了 "This frame is a (suspected) retransmission" 提示。把过滤器换成 tcp.analysis.fast_retransmission 则没有任何结果，说明这三条重复 ACK 并没有触发快重传。

3.8 吞吐量

本次连接在 TCP Conversations 统计中的结果是：客户端→服务器 111 个报文、152566 字节；服务器→客户端 33 个报文、724 字节；会话从 1.976791 s 开始，持续 2.3166 s。因此平均吞吐量为：

```
吞吐量 = 152566 字节 / 2.3166 秒 ≈ 65858 字节/秒 ≈ 64.3 KB/s ≈ 527 kbit/s
```

如果把连接的建立与关闭段排除，只看数据真正在传输的时间窗（2.233376 s 到 3.263068 s，共 1.029692 s），则瞬时吞吐量可达 152566 / 1.029692 ≈ 148166 字节/秒 ≈ 144.7 KB/s ≈ 1.19 Mbit/s。两个数值差了将近 2.25 倍，原因是总时长里包含了约 0.256 s 的握手等待与约 1.03 s 的四次挥手等待。这也说明：对于短传输，握手与关闭的固定开销占比很大，报告吞吐量时必须说明所取的统计口径，否则数字没有可比性。

3.9 时间-序号图与拥塞控制

用统计→TCP 流图形→时间序列(Stevens) 画出客户端→服务器方向的图（图 4）。图中每个点代表一个数据段，横轴是发送时刻，纵轴是该段的序号。同一时刻竖直排开的一串点表示这些段是背靠背连续发出的，这一串点的长度就是当时拥塞窗口允许发送的数据量。

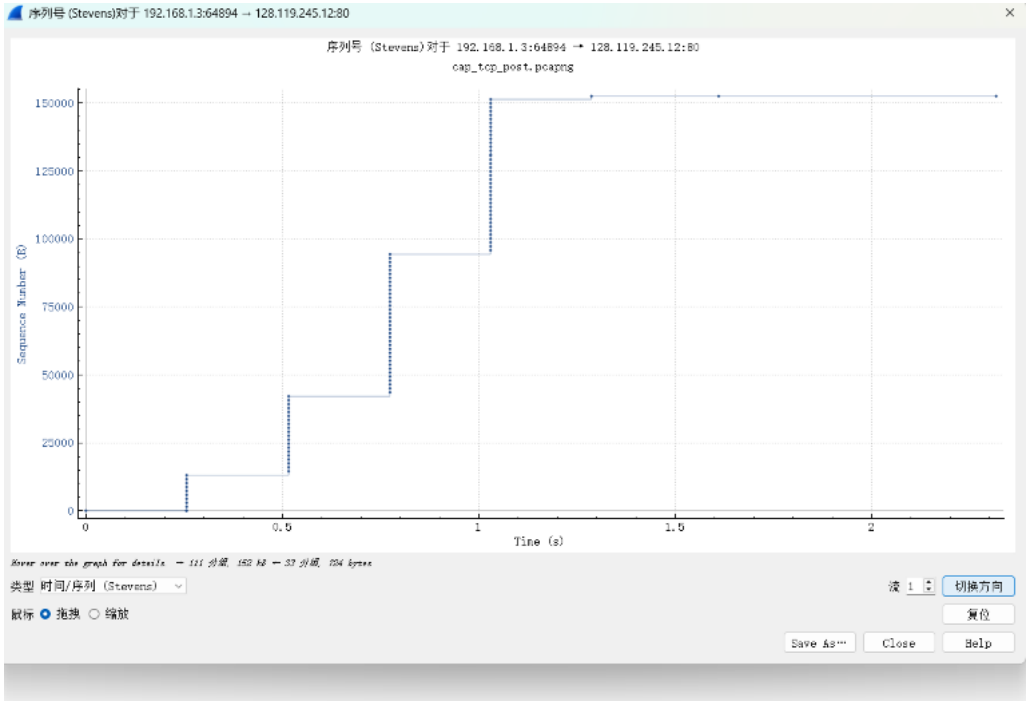


图 4 Time-Sequence-Graph(Stevens)：客户端到服务器的序号-时间图

把图中每一级台阶的段数与字节数统计出来，得到下表。这张表比图本身更能说明问题：

表 3 五轮发送突发的规模

突发序号	起始时刻 (s)	段数	字节数	起始序号	结束序号
1	2.233376	10	14520	1	14521
2	2.492831	20	29040	14521	43561
3	2.750729	36	52272	43561	95833
4	3.006572	25	35240	95833	131073
5	3.006897	15	21494	131073	152567

前三级台阶的规模是 10 段 → 20 段 → 36 段，第一级到第二级正好翻倍，第二级到第三级也接近翻倍（1.8 倍）；每一级台阶之间的时间间隔约为 0.257 s，恰好是一个 RTT。这正是慢启动的教科书式特征：发送方每收到一个确认就把拥塞窗口加一个 MSS，于是一个 RTT 内窗口翻倍，可以连续发出的数据量随之指数增长。第一轮只有 10 段是因为初始拥塞窗口（initcwnd）被设为 10 个 MSS，并不是从 1 个 MSS 开始的，这是 RFC 6928 之后 Linux 的默认行为，与课本里"从 1 开始"的理想描述不同。

第三轮之后曲线突然停住，最后两轮（25 段与 15 段）几乎发生在同一时刻（3.0066 s）。这并不表示进入了拥塞避免：整个文件只有 152567 字节，按每 RTT 翻倍的节奏到第四轮就快传完了，发送方在最后把剩余数据一次性交了出去。也就是说，本次实验根本没有跑到拥塞避免阶段，自然也就观察不到"每 RTT 线性加 1"的斜坡。若要观察拥塞避免，需要传输一个远大于带宽时延积的文件，或者人为制造丢包让 cwnd 越过慢启动阈值。

四、实验问题解答

表 4 TCP 实验问题逐条解答

序号	问题	解答与依据
1	承载 alice.txt 传输的客户端 IP 与端口是什么？	192.168.1.3，源端口 64894。该连接在抓包文件中编号为 tcp.stream 1，与之前在 stream 0 中取回复页面的连接是两条独立连接。
2	gaia.cs.umass.edu 的 IP 地址与收发端口是什么？	128.119.245.12，TCP 端口 80。该地址也出现在 DNS 查询结果中（见实验三），可互相印证。
3	本机抓到的这条连接中，谁是发送方？	本机是发送方（数据方向为客户端→服务器），因为本次操作是上传 152138 字节的 alice.txt，抓包中 111 个报文、152566 字节来自本机，服务器只回了 724 字节的响应页。
4	SYN 段的序号是多少？靠什么标识它是 SYN？	相对序号 Seq=0（绝对序号 1903602222），且首部中没有有效载荷（Len=0）。标识依据是标志位中的 SYN 位被置 1（Wireshark 显示为 [SYN]），同时 ACK 位为 0；此外它携带了 MSS=1460、WS=256、SACK_PERM=1 等只在建立阶段出现的选项。
5	SYNACK 的序号与确认号是多少？确认号怎么得来的？靠什么标识它是 SYNACK？	Seq=0，Ack=1。确认号 1 = 收到的 SYN 的序号 0 + 1，即"我期待你下一个字节的序号是 1"，而 SYN 本身占一个序号，所以加 1。标识依据是 SYN 与 ACK 两个标志位同时置 1（Wireshark 显示为 [SYN, ACK]）。
6	承载 HTTP POST 命令的段的序号是多少？	相对序号 Seq=1（绝对序号 1903602223），长度 1452 字节。它是连接建立后的第一个数据段，载荷开头为 "POST /wireshark-labs/lab3-1-reply.htm HTTP/1.1"。
7	前六个数据段的序号、发送时刻、确认时刻、RTT 与 EstimatedRTT 分别是多少？	序号依次为 1、1453、2905、4357、5809、7261；六个段在同一时刻 2.233376 s 背靠背发出；它们被同一个 ACK（第 15 号，ack=8713=6×1452+1）在 2.492751 s 一并确认；故六个 SampleRTT 都是 0.259375 s。EstimatedRTT 取初值 0.259375

序号	问题	解答与依据
		后按 0.875×旧值+0.125×新值 递推，由于样本与初值相同，六个值都保持 0.259375 s。（若继续用到后续 ACK 的样本 0.257804/0.255731/0.256171/0.255064，则得 0.259179、0.258748、0.258426、0.258005。）
8	前六个 TCP 段的长度各是多少？	全部为 1452 字节，即 MSS=1452。1452 = 1500（以太网 MTU）− 20（IP 首部）− 28（TCP 首部 20 + 时间戳选项 12），这也解释了为什么服务器通告的 MSS 是 1452 而客户端通告的是 1460：客户端在 SYN 中带了时间戳选项，服务器据此把 MSS 下调了 8 字节。
9	接收方通告的最小可用缓冲区是多少？缓冲不足是否限制了发送方？	缩放后的最小可用窗口为 72192 字节（第 39 号报文，window_size_value 564 × 缩放因子 128）；SYNACK 中的 64240 是尚未启用缩放时的原始通告值。窗口从未降到 0，且每轮突发（最大 52272 字节）都小于当时可用窗口，因此接收缓冲没有限制发送方，真正的限制是拥塞窗口。
10	抓包中是否存在重传？你是如何判断的？	有，但只有两个，且都在连接关闭阶段：第 142 号（本机 FIN，seq=152567）与第 144 号（服务器 FIN，seq=725）。判断方法：（1）用过滤器 tcp.analysis.retransmission 筛出；（2）Info 列直接标注 [TCP Retransmission]，行背景为深色；（3）报文序号与前一个报文完全相同且 Len=0，说明发送了重复的序号。数据段无任何重传，说明链路质量良好。
11	接收方通常一次确认多少数据？是否存在"每两个段确认一次"的情况？	观察相邻 ACK 的确认号增量：多为 2904 字节（正好 2 个段）或 5808 字节（4 个段），最大 14520（10 个段），最小 1452（1 个段）。存在明显的"每两个段一次 ACK"的情况，这正是课本表 3.2 所述延迟确认策略的体现；数据密集到达时一次 ACK 会累积确认更多段。
12	TCP 连接的吞吐量是多少？如何计算的？	按整条连接的时长计算：152566 字节 / 2.3166 秒 ≈ 65858 B/s ≈ 64.3 KB/s ≈ 527 kbit/s。若只取数据实际传输的时间窗（2.233376→3.263068 s，1.029692 s），则为 152566 / 1.029692 ≈ 148166 B/s ≈ 144.7 KB/s ≈ 1.19 Mbit/s。字节数取自统计→会话→TCP 中该连接的统计结果，时间取自第一个报文与最后一个报文的时间差。
13	在时间-序号图上，慢启动从何处开始、到何处结束？拥塞避免从何处接管？实测与理想模型有何差别？	慢启动从 2.233376 s 开始，此时发送方一次性发出 10 个段（初始拥塞窗口 10 MSS）；此后每一轮突发都比上一轮大约翻倍（10→20→36 段），台阶间隔约 0.257 s 正好是一个 RTT。到 3.0066 s 附近传输结束，曲线被数据量截断，本次实验未进入拥塞避免阶段，因此图上没有出现线性增长的斜坡。与理想模型的差别有三点：（1）理想模型从 1 个 MSS 起步，实测因 initcwnd=10 而从 10 起步；（2）理想曲线是平滑的指数曲线，实测是离散的台阶，因为窗口只在一个 RTT 的边界上才更新一次；（3）理想模型中接收窗口是无限的，实测窗口虽大但有限，只是本次没有成为约束。
14	用你自己抓的抓包数据重复上述两问，结论是否一致？	一致。本报告第 3.9 节用的就是本机上传 alice.txt 时抓到的真实数据，得到的台阶结构（10/20/36 段，间隔约 1 个 RTT）与指导书所给示例的定性特征相同，差异仅在于具体数值：示例的初始窗口更小、传输文件更大，因此在其图上能看到更长的慢启动段甚至拥塞避免段。

五、实验总结

5.1 遇到的问题与解决办法

表 5 TCP 实验中的常见问题与处理

问题现象	原因分析	解决办法
下载 tcp-ethereal-trace-1 后，序号-时间图的发送方向是服务器	该抓包文件是"服务器→客户端"的下载场景	改为本机上传 alice.txt，使发送方为本机，与第 13 题的观察意图一致
六个数据段算出的 RTT 完全相同，怀疑算错	延迟确认把六个段合并成一个 ACK 确认	改为按"每轮突发"统计 RTT，并用整个连接的所有 ACK 样本检验
ACK 的确认号出现"后退"（63889→60985）	这些 ACK 时间戳相同，是被批量发出后在链路上重排	以时间戳和确认号最大值判断真实进度，不要按列表顺序推断
时间-序号图几乎看不到台阶，只有一条竖线	图形窗口默认方向可能是服务器→客户端，且纵轴量程会自动缩放	点「切换方向」把方向切到客户端→服务器，必要时在图上拖动放大
MSS 与课本不一致（1452 而非 1460）	本机 SYN 带了时间戳选项，服务器据此下调 MSS	在报告中说明 1452 = 1500−20−28，而不是当成错误

5.2 收获与体会

这次实验让我对"TCP 是字节流而不是报文流"有了具体感受。课本上说序号按字节编号、确认号表示"下一个期望的字节"，平时只是记住这句话；当看到 ack=8713 而它在数值上等于 6×1452+1 时，才真正理解为什么六个报文可以共用一个确认——因为确认的对象是字节区间，而不是报文。同样地，那个"确认号后退"的现象如果不看时间戳，很容易被误判为 TCP 出错，这提醒我抓包分析必须结合多个字段交叉验证。

第二个收获是拥塞控制的"可视化"。以前对慢启动的理解停留在"每 RTT 翻倍"这句话上，而时间-序号图把这句话变成了看得见的台阶：10 段、20 段、36 段，每一级之间恰好隔一个 RTT。更让我意外的是实验根本没跑到拥塞避免——152 KB 的文件在 1 秒内就传完了，慢启动的指数增长还没撞到任何边界就已经结束。这说明在真实网络里，绝大多数短连接都活在慢启动阶段，这也是为什么 CDN 要把服务器部署得离用户更近：缩短 RTT 就等于让慢启动更快爬升。

第三个收获来自吞吐量的两种算法。同一份数据，按整条连接算是 64.3 KB/s，按纯传输窗口算却是 144.7 KB/s，差了两倍多。起初我以为是自己算错了，把握手和四次挥手的等待时间逐项减掉之后才明白：对于只持续一秒多的传输，握手和关闭的固定开销占比极大。这让我以后再看到"某协议吞吐量多少"的说法时，会先问一句"口径是什么"——是端

到端还是纯传输、是否含建连、是否含应用层开销。这种对度量口径的敏感，可能比记住某个具体数字更有用。